

Software Engineering II

Architekturmuster

Mathis Neunzig
ABAP Developer at SAP
mathis.neunzig@gmail.com



Disclaimer

- Diese Präsentation sowie alle weiteren Materialien dieser Vorlesung sind kein Ersatz für Literatur und können nicht als Quellen für wissenschaftliche Arbeiten genutzt werden.
- Diese Präsentation sowie alle weiteren Materialien dieser Vorlesungen dürfen nur von den Studierenden des Kurses WI 2023 SE B zur Nachbereitung der Vorlesung genutzt werden. Eine kommerzielle Nutzung und Weitergabe ist verboten.
- Alle Präsentation sowie alle weiteren Materialien werden vor Beginn der jeweiligen Vorlesung hochgeladen.
- Zur besseren Lesbarkeit wird auf die gleichzeitige Nutzung von weiblichen & männlichen Begriffen verzichtet. Es wird das generische Maskulinum verwendet. Dabei sind weibliche und sonstige Formen des Wortes gleichwertig mitgemeint. (Beispiel: statt "Kund:innen" → „Kunden“)

Architekturmuster

Was wir in der VL machen
ist violett markiert!

Strukturmuster

- Layered Architecture
- MVC / MVP / MVVM
- Pipe-and-Filter
- Microkernel (Plug-in Architecture)
- Modular Monolith
- Component-Based Architecture
- Self-contained Systems (SCS)
- Blackboard Architecture

Verteilungsmuster

- Client-Server
- Microservices
- Service-Oriented Architecture (SOA)
- Peer-to-peer (P2P)
- Event-Driven Architecture (EDA)
- Space-Based Architecture
- Shared Nothing Architecture
- Service Mesh

Verantwortlichkeits-trennungsmuster

- Domain-Driven-Design (DDD)
- Command Query Responsibility Segregation (CQRS)
- Hexagonal Architecture
- Onion Architecture
- Clean Architecture
- Screaming Architecture

Deployment- & Betriebsmodelle

- Serverless Architecture
- Containerized Architecture
- Backend for Frontend
- API Gateway Pattern
- Sidecar Pattern
- Multi-Tenant Architecture
- Twelve-Factor App
- Immutable Infrastructure

Integrationsmuster

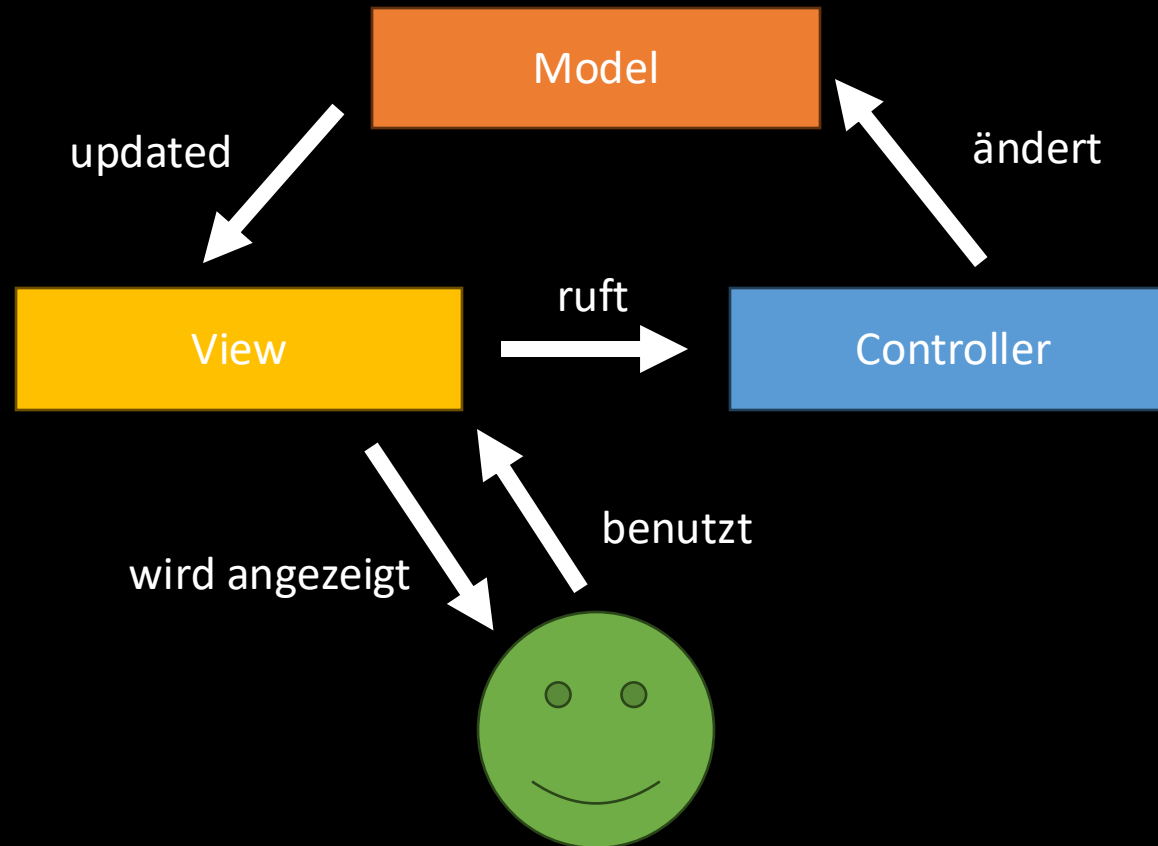
- Adapter
- Facade
- Message Bus & Message Queue
- Publish-Subscribe
- Saga Pattern
- Anti-Corruption Layer
- Gateway
- Event-Carried State Transfer
- Shared Database

Kommunikationsmuster (nicht offiziell Teil von Architekturmustern): REST, GraphQL, SOAP, Odata, Websocket, Remote Procedure Call, Graph-based API, JSON-RPC, XML-RPC

MVC, MVP, MVVM

- GUI-Logik wird schnell complex Logik, UI, Daten
- Trennung notwendig für:
 - Testbarkeit
 - Wartbarkeit
 - Wiederverwendbarkeit
 - Teamarbeit (Frontend/Backend)
- „Separation of Concerns“ als Schlüsselprinzip
- Ursprung: Smalltalk (1970er), später in Web- und Desktop-entwicklung übernommen

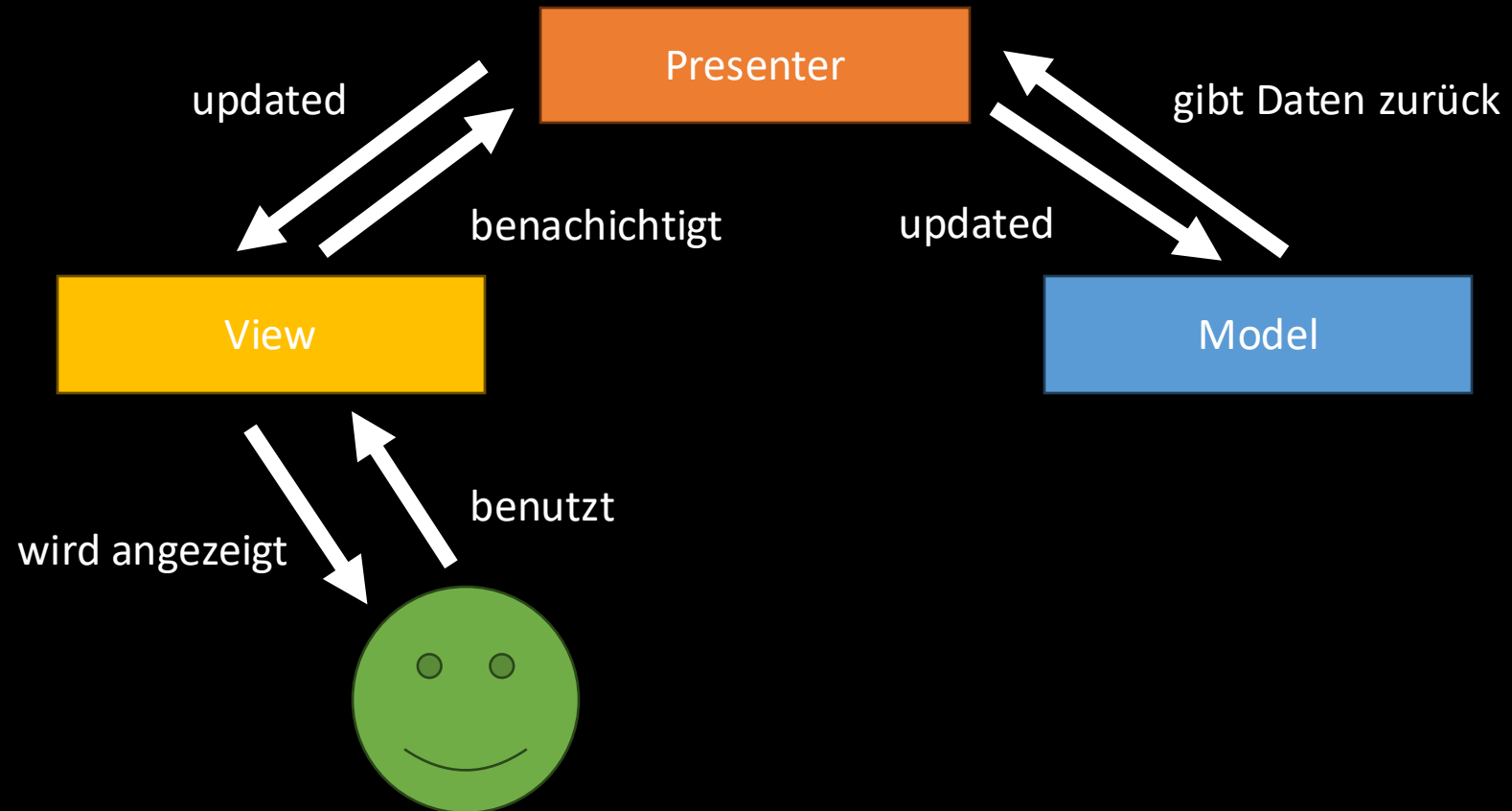
MVC



MVC

- Model View Controller
- Erfinder: Trygve Reenskaug, Xerox PARC, 1979
- View = Ausgabe, Benutzeroberfläche
- Controller = Eingabe, Logik
- Model = Daten
- Fun fact: MVC wurde ursprünglich für Desktop-GUIs entwickelt
- Beispiel: Java SpringBoot
 - Controller = REST-Endpunkte mit Logik
 - Model = Entities
 - View = HTML mit Thymeleaf (oder ausgelagert)

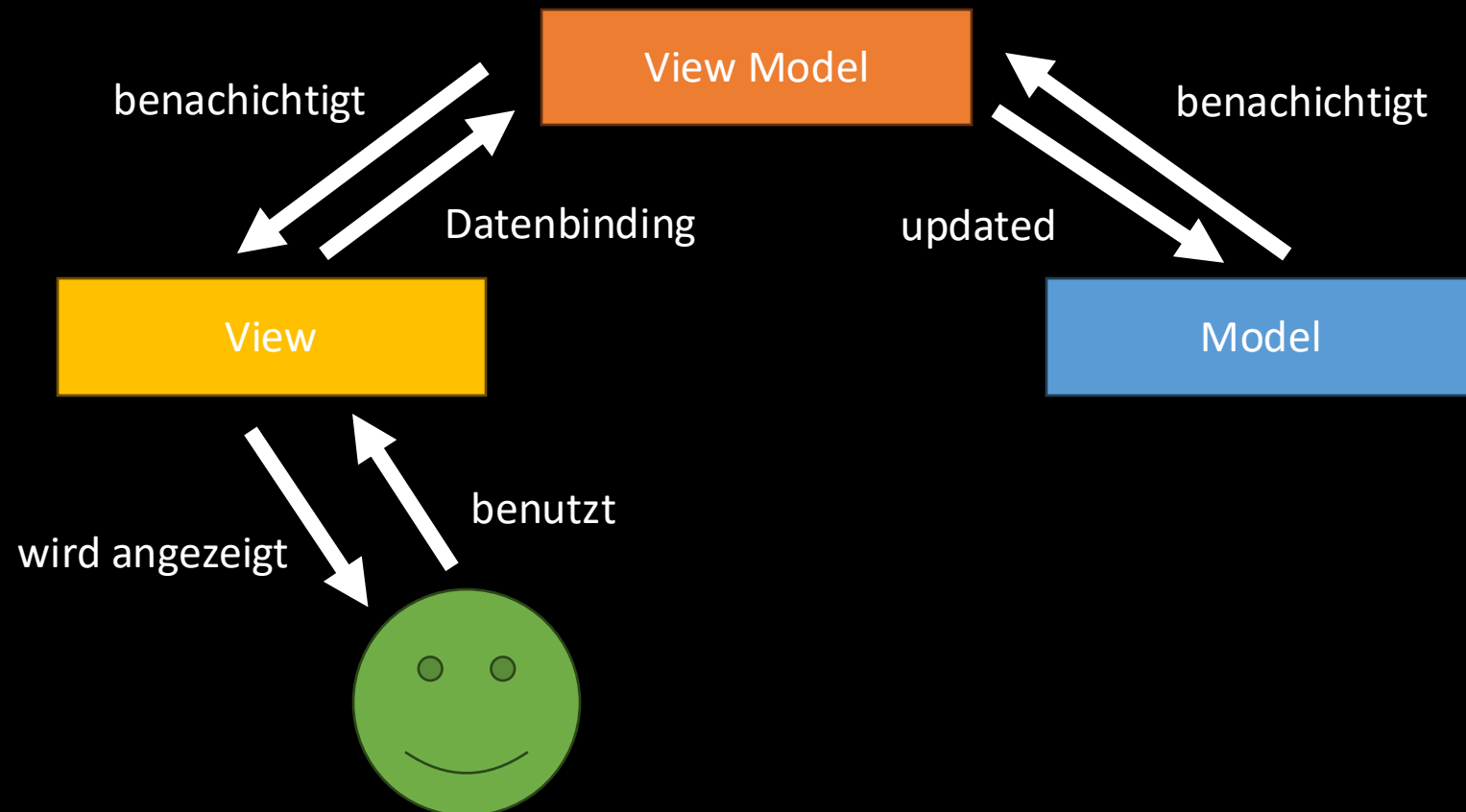
MVP



MVP

- Model View Presenter
- Sonderform vom MVC
- Keine Kommunikation zwischen Model und View!
- Presenter bekommt ausschließlich Events vom View
 - ButtonClick
 - RightMouseClicked
- Presenter testbar ohne UI
- Beispiel: Google Web Toolkit oder ältere Android-Apps

MVVM



MVVM

- Model View View-Model
- Datenbinding (Zwei-Wege-Datenbindung)
- View-Model stellt dar, was im View gebraucht wird
- Beispiel: Angular (ngModel) oder React (State & Hooks)

MVC vs MVP vs MVVM

Kriterium	MVC (Model-View-Controller)	MVP (Model-View-Presenter)	MVVM (Model-View-ViewModel)
Trennung der Komponenten	Teilweise – View ruft Controller auf	Klar getrennt – View ist passiv	Sehr klar – ViewModel kapselt Logik & Status
Logik liegt in...	Controller	Presenter	ViewModel
UI ist...	aktiv – enthält etwas Logik & ruft Controller direkt	passiv – nur Eventweitergabe	datengebunden – UI verändert sich automatisch
Model kennt View?	Nein	Nein	Nein
View kennt Model?	Manchmal (über Controller)	Nein – nur über Presenter	Nein – nur über ViewModel
Data Binding	Nein oder manuell	Nein	Ja – automatische Zwei-Wege-Bindung möglich
Testbarkeit	mittel – Logik oft verstreut	hoch – Presenter kann isoliert getestet werden	hoch – ViewModel isoliert testbar, View „dumm“
Komplexität der Struktur	gering bis mittel	mittel	höher (besonders bei Data Binding und State)
Typische Verwendung	Webframeworks (Spring MVC, Rails)	Android (ältere Patterns), WinForms	Angular, React (mit Hooks/State), WPF
Zugrundeliegendes Prinzip	Eventweiterleitung + Zustandsaktualisierung	Passive View + Presenter steuert komplett	Data Binding + Observable State
Erlernbarkeit	leicht	mittel	komplexer (besonders Bindings)
Beispiel (aus JS-Projekt)	Controller.handleAdd() ruft model.addTo() und view.render()	Presenter.handleAdd() steuert alles, View nur Anzeige	ViewModel.addTo() aktualisiert State, View reagiert automatisch

Microkernel

- Trennung von
 - Kernfunktionalität (Microkernel)
 - erweiterbarer Logik (Plug-ins)
- Fokus: Modularität, Erweiterbarkeit, Anpassbarkeit
- Auch bekannt als Plug-in-Architektur
- Der „Kernel“ stellt Grunddienste bereit – Plug-ins bauen darauf auf

Microkernel

- Beispiele
 - Webbrowser wie Chrome und Chrome-Extensions
 - Betriebssystem-Kernel mit Treibern
 - Entwicklungsumgebungen wie VS Code mit Extensions
 - SAP ERP mit diversen Modulen, die installiert werden können

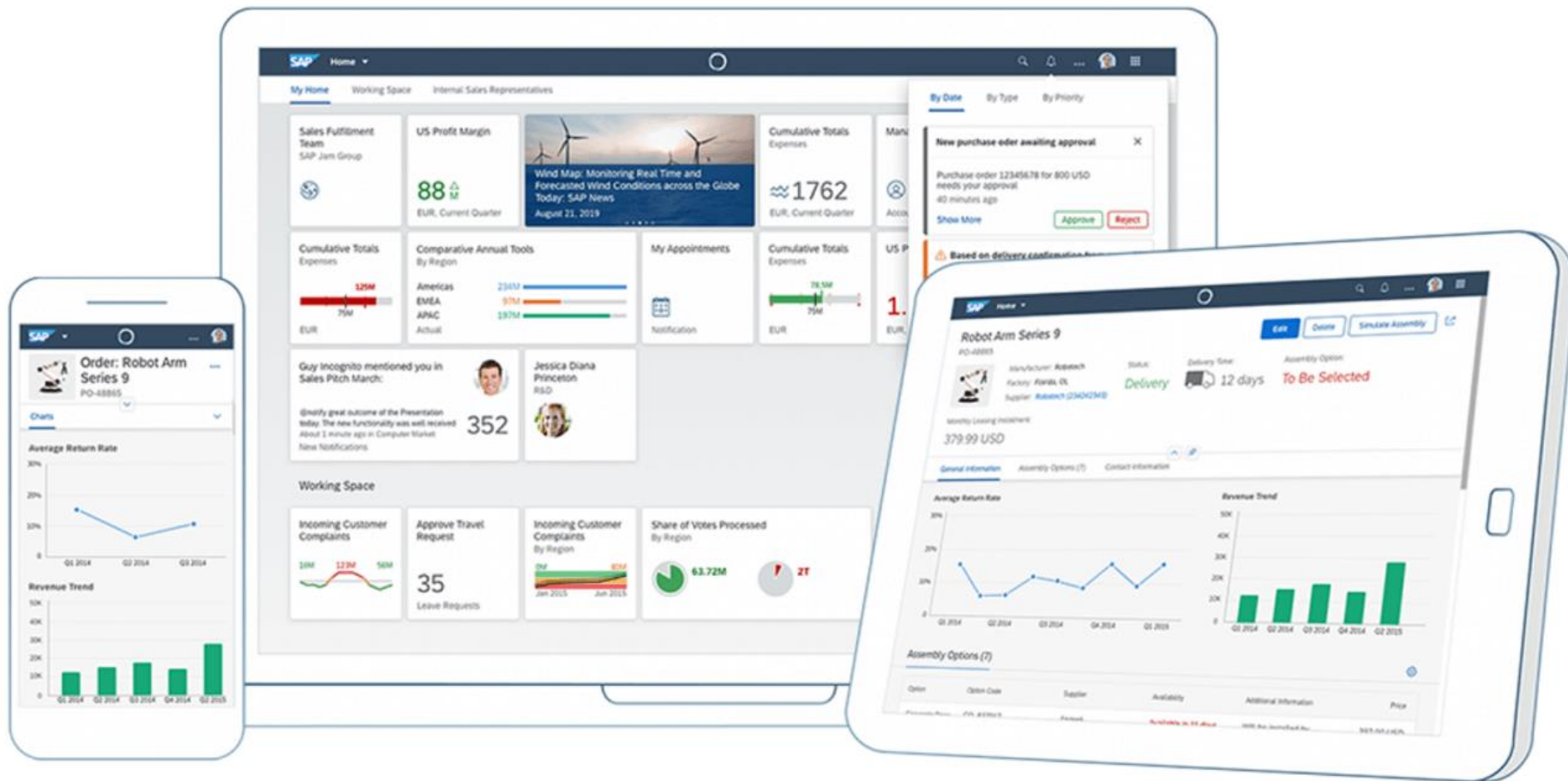
Microkernel

- Aufbau
 - Microkernel
Enthält Basisdienste (z. B. Eventsystem, Logging, APIs)
 - Plug-ins
Erweiterungen, die über definierte Schnittstellen eingebunden werden
 - Plug-in-Registry / Loader
Erkennt, lädt und aktiviert Plug-ins dynamisch

Microkernel

- Vorteile
 - Hohe Erweiterbarkeit
 - Klare Trennung von Kern & Erweiterung
 - Flexibel für unterschiedliche Kunden / Anwendungsfälle
 - Gute Wartbarkeit & Testbarkeit einzelner Plug-ins
- Geeignet für
 - Produktlinien (Software mit Varianten)
 - IDEs, Compiler, Frameworks
 - Systeme mit Third-Party Extensions

Microkernel



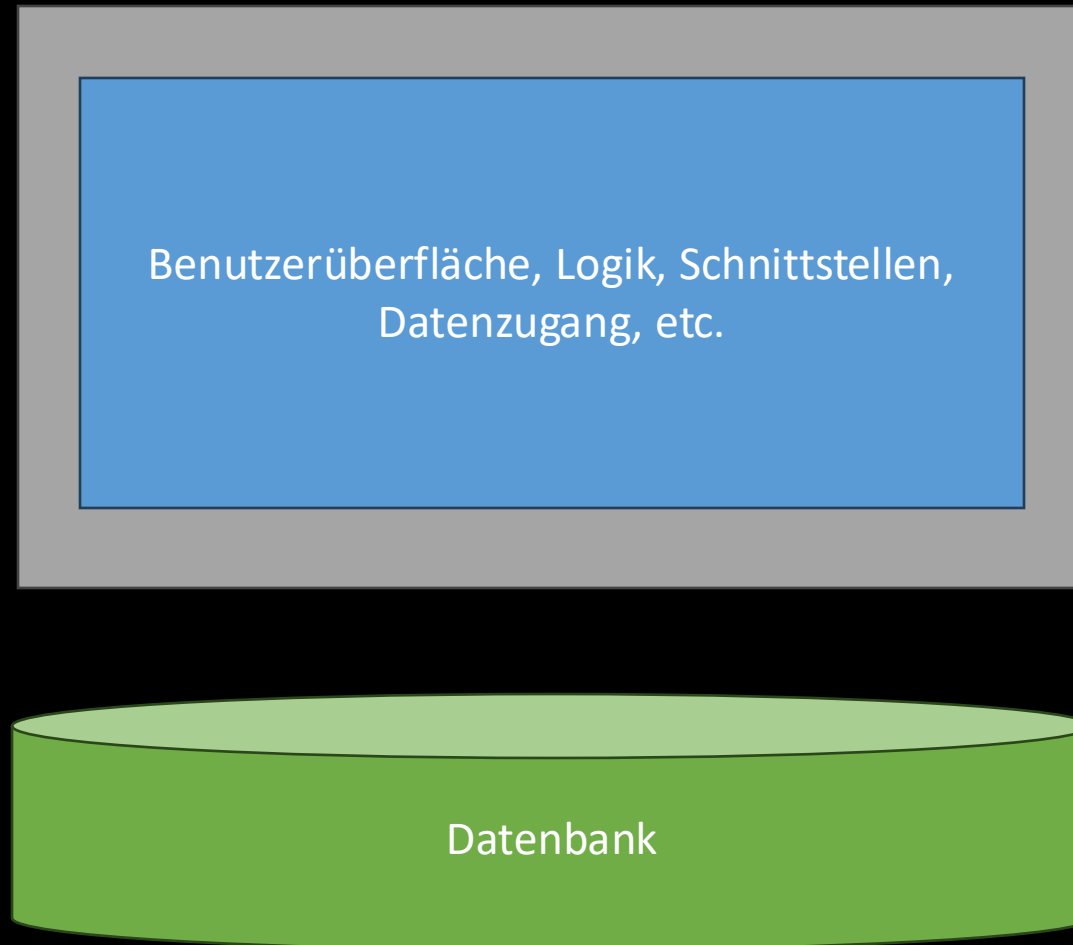
Übung: Microkernel

- Voraussetzung: GitHub-Konto
- Aufgabe: Schreibe ein Consumer- und ein Provider-Plugin
- Siehe GitHub
- 5 Punkte
- Abgabe: am Abend vor der nächsten Vorlesung (als Pull Request)

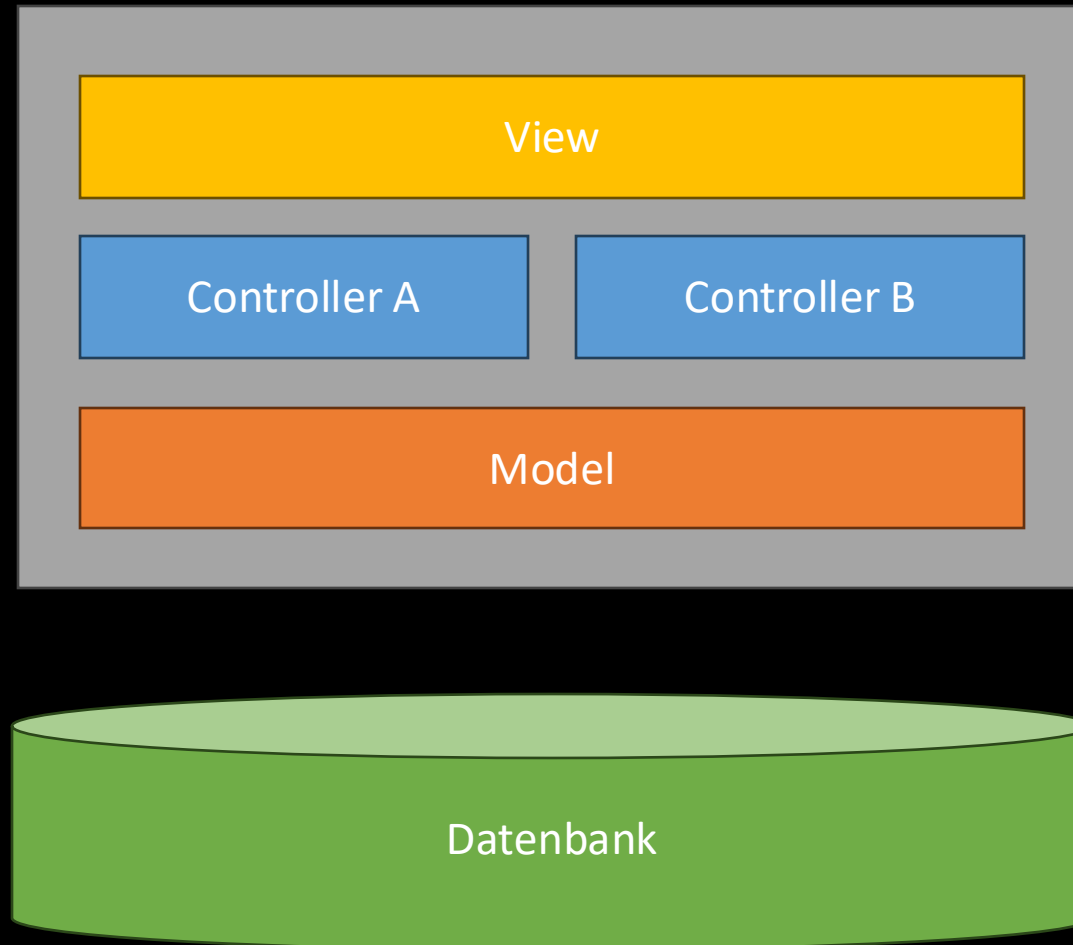
Monolith

- Eine Softwarearchitektur, bei der alle Funktionen in einer einzelnen Anwendung gebündelt sind
- Beispiel: Frontend, Backend und Datenzugriff in einem Deployable
- Änderungen erfordern oft Anpassungen im ganzen System
- Läuft typischerweise als einziger Prozess / Application Server
- Beispiele
 - Klassische Webshops oder ERP-Systeme
 - WordPress, Magento (in ursprünglicher Form)
 - Viele ältere Unternehmensanwendungen („Fat Clients“)
 - Desktop-Anwendungen (z.B. Microsoft Word als eigenständige Applikation)

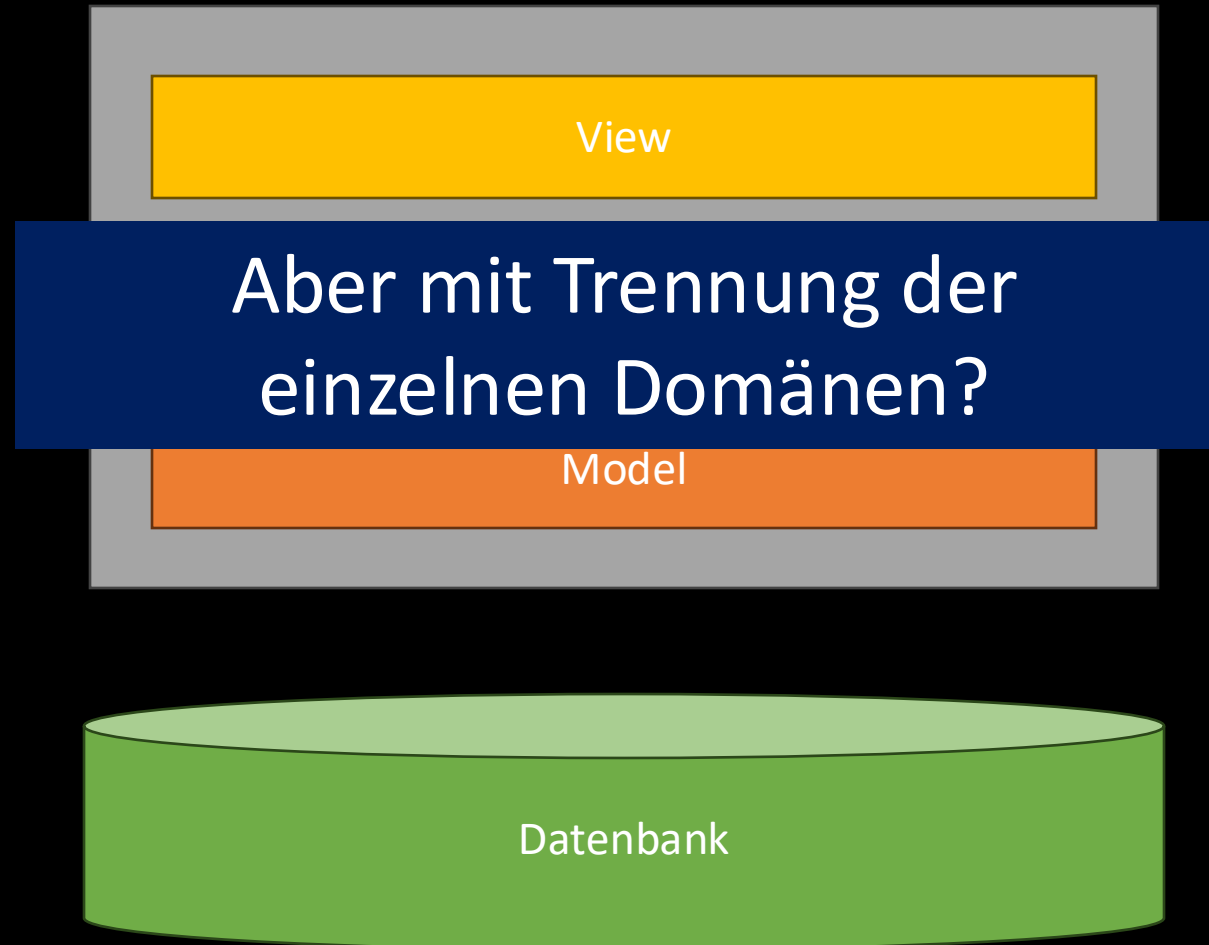
Monolith



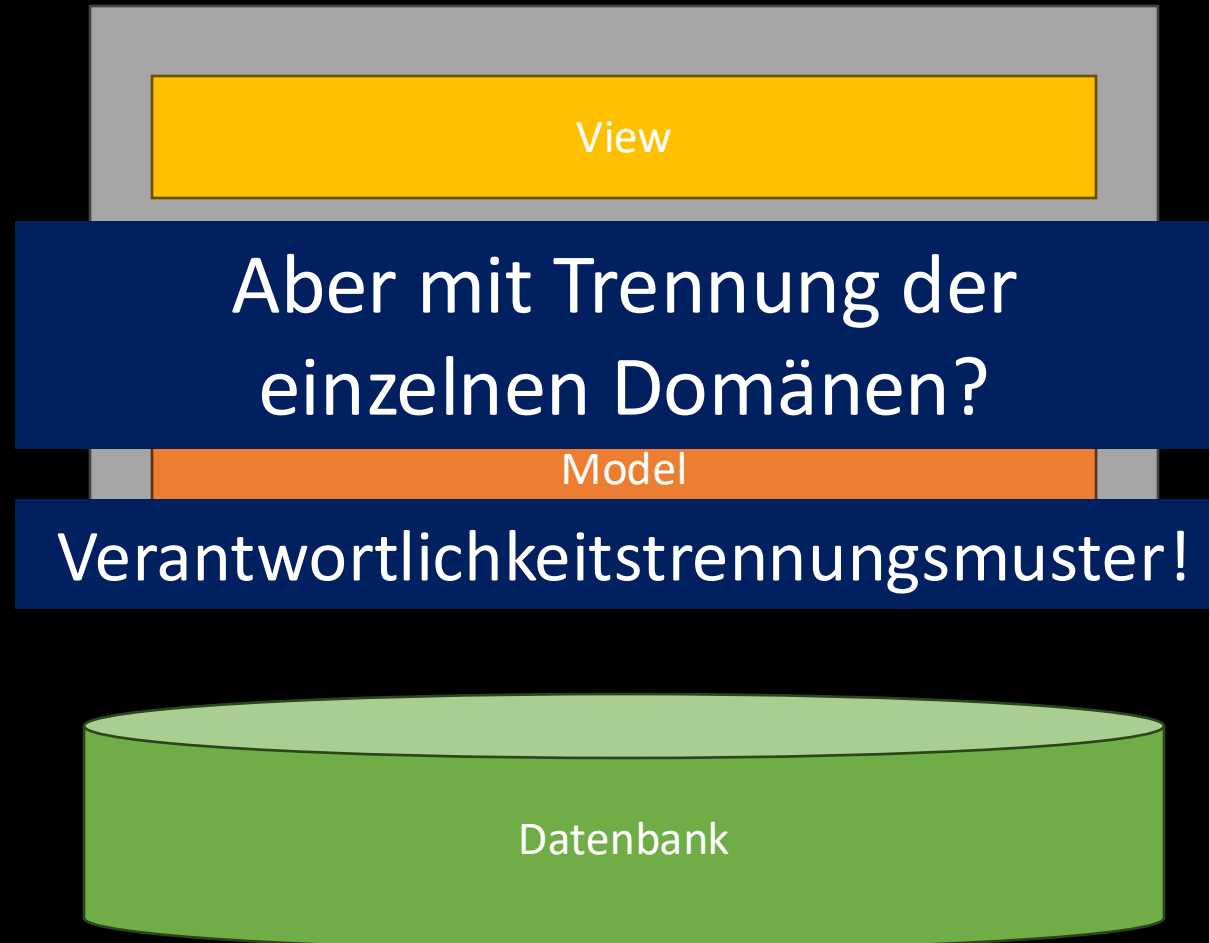
Monolith



Monolith



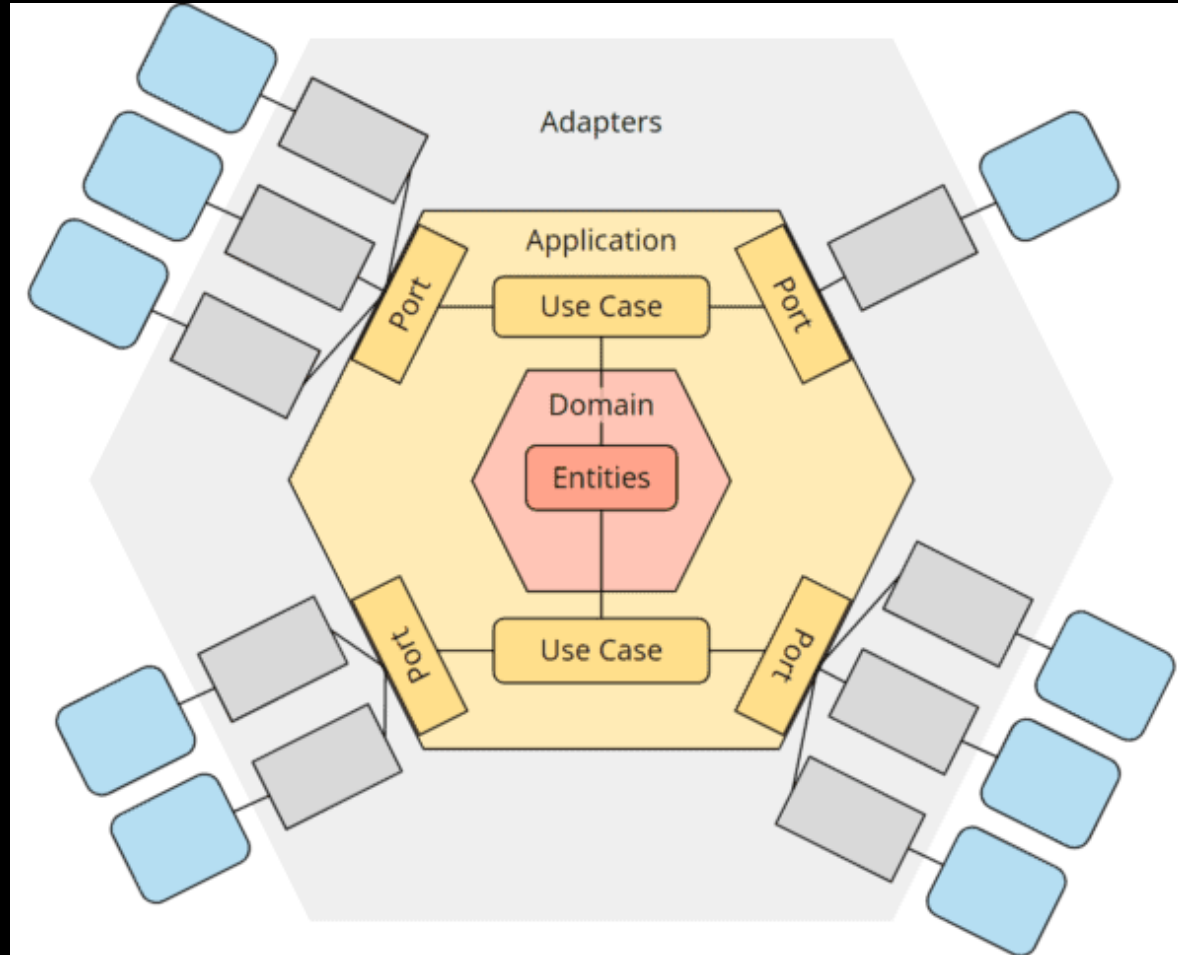
Monolith



Hexagonal Architecture

- “Technologie-unabhängiger Monolith”
- Trennung von Kernlogik und äußeren Schnittstellen
- Zentraler Kern, andere Funktionen sind ausgelagert
- Kommunikation mit Außenwelt über Ports und Adapters
- Vorteile
 - einfache Austauschbarkeit von UI
 - Datenbanken
 - oder externen Systemen
- Sehr gut geeignet, um Abhängigkeiten umzukehren und Testbarkeit zu erhöhen

Hexagonal Architecture



Beispiel: Hexagonal Architecture

Element	Beispiel	Funktion
Application Core	core/...	Enthält fachliche Logik, aber hat keine Abhängigkeiten nach außen
Ports	ports/...	Definition der Kommunikation mit der Außenwelt
Inbound Adapter	adapters/ExpressAdapter	Implementierung der Ports & Kommunikation zwischen Kern und Außenwelt
Outbound Adapter	adapters/ExternalProductAdapter	
External System	python-product-service/...	Technologieunabhängiger Service bzw. Auslagerung der Logik & Datenhaltung

Monolith

✓ Pro

- Einfaches Deployment (nur eine App starten)
- Weniger Overhead für Kommunikation
- Geringere Komplexität für kleine bis mittlere Systeme

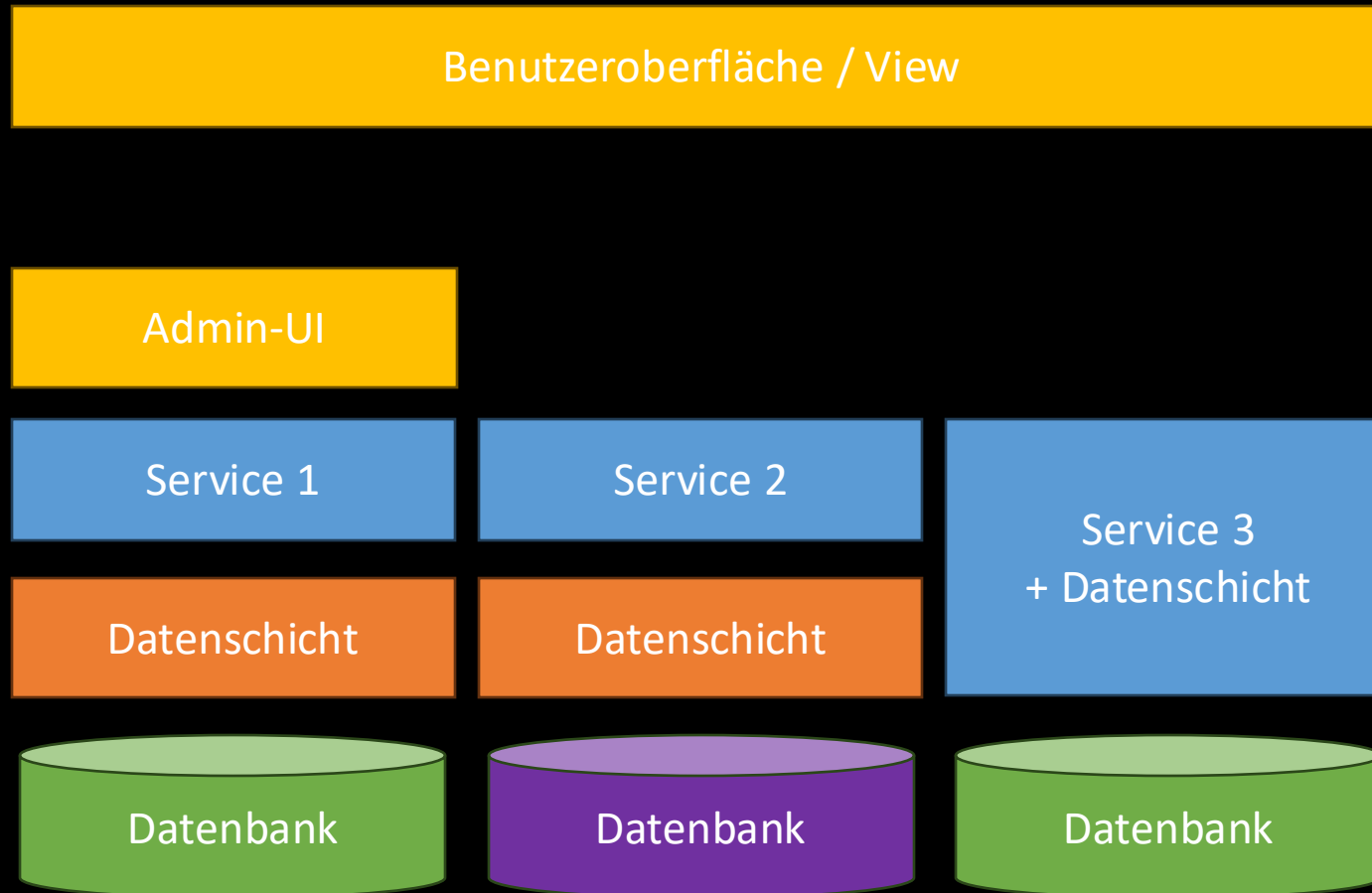
✗ Contra

- Skalierungsprobleme bei wachsendem System
- Änderungen können viele Teile beeinflussen (hohe Kopplung)
- Risiko bei Ausfall: gesamtes System betroffen!

Microservices

- Architekturansatz: Anwendung besteht aus vielen unabhängigen Services
- Jeder Service hat eigene Verantwortung („Single Responsibility Principle“)
- Kommunikation oft über APIs oder Messaging (z.B. RabbitMQ)
- Jeder Service kann unabhängig entwickelt, deployt und skaliert werden
- Jedes Team kann einen eigenen Service betreuen („Team Ownership“)
- Beispiele
 - Amazon: „Product Catalog Service“, „Recommendation Service“
 - Netflix: „User Management“, „Streaming Service“, „Payment Service“
 - E-Commerce: „Bestellservice“, „Zahlungsservice“, „Versandservice“

Microservices



Microservices

✓ Pro

- Bessere Skalierbarkeit
- Schnellere Entwicklung in kleinen Teams
- Technologievierfalt pro Service möglich
(Jeder Service kann eine andere Technologie haben)

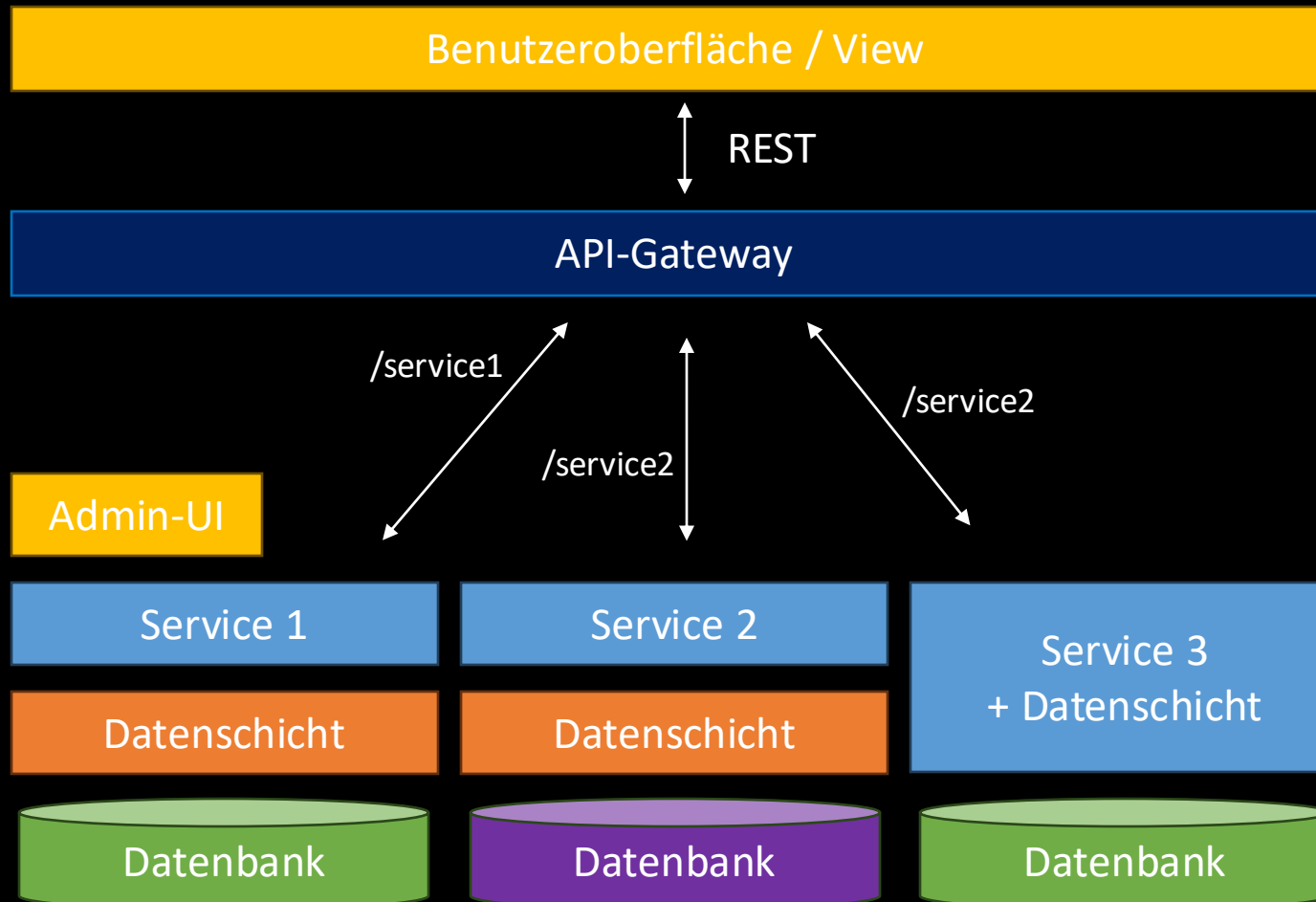
✗ Contra

- Hohe Komplexität: Monitoring, Logging, verteilte Transaktionen
- Netzwerk-Overhead
- Daten könnten inkonsistent werden

API-Gateway

- Zentrale Schnittstelle für alle Anfragen von außen
- Aufgaben
 - Routing der Anfragen an richtige Microservices
 - Authentifizierung & Autorisierung
 - Load Balancing & Caching
 - Schutz vor Überlastung (Rate Limiting)
- Beispielsoftware
 - Kong
 - NGINX
 - AWS API Gateway

API-Gateway



API-Gateway

```
// Product Service
app.use('/products', createProxyMiddleware({
  target: 'http://localhost:3001',
  changeOrigin: true,
  pathRewrite: { '^/products': '' },
}));

app.listen(PORT, () => {
  console.log(`API Gateway running at http://localhost:${PORT}`);
});
```


Monitoring

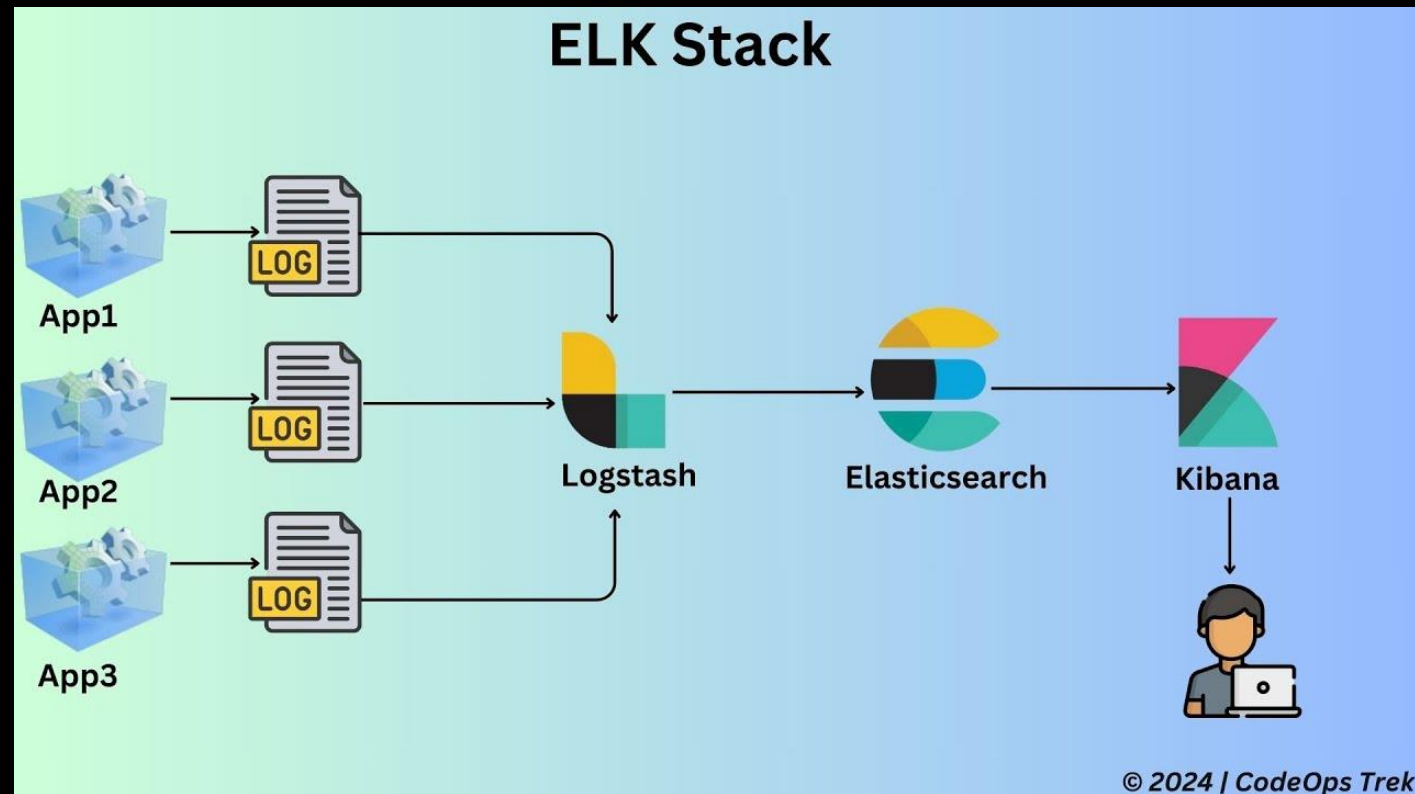
- Tools zur Überwachung von Metriken
 - CPU-Auslastung
 - RAM
 - Latenz
- Beispiele
 - Grafana
 - Prometheus
- Vorteile
 - Fehler Schneller erkennen
 - Health-Checks
 - Engpässe und Bottlenecks erkennen und fixen

Monitoring



Logging

- ELK-Stack
 - Elasticsearch
 - Logstash
 - Kibana



Serviceorientierte Architektur (SOA)

- Architekturansatz aus den 2000ern
- Services als lose gekoppelte Komponenten
- Fokus auf Wiederverwendbarkeit, Interoperabilität und Integration
- Nutzung von ESB (Enterprise Service Bus) zur Koordination
- Standardprotokolle wie SOAP oder auch REST

SOA vs. Monolith

SOA

- Lose gekoppelte Services
- Oft über EBS implementiert
- Erlaubt Integration bestehender Systeme (z.B. SAP + ...)
- Flexibler bei großen Umgebungen

Monolith

- Eine große Anwendung
- Enge Kopplung aller Bestandteile
- Einfach bei kleineren Systemen
- Single Point of Failure

SOA vs. Microservices

SOA

- Unterschiedliche Services
- Oft zentralisiertes Management

Microservices

- Unterschiedliche Services
- Oft dezentral & Container-based
- Meist Schneller und agiler in Entwicklung & Deployment

Serviceorientierte Architektur (SOA)

✓ Pro

- Integration von Altsystemen
- Wiederverwendbare Services für viele Anwendungen
- Gut für große Unternehmen mit komplexen IT-Landschaften

✗ Contra

- Hohe Komplexität durch EBS
- Schwerfälliger als Microservices
- Overhead durch standardisierte Kommunikation, z.B. mit SOAP

Danke!

Mathis Neunzig
mathis.neunzig@gmail.com

